Prishanga Bora Follow

Aug 21, 2022 · 6 min read · Listen

Save

[Tutorial] Training PaddleOCR Scene Text Recognition in the Wild with Custom Dataset

Through this article I would be training both detection and recognition modules of PP-OCR to create a full fledged scene text recognition system.

This part would involve transfer learning and training from scratch for both modules on Scene text dataset consisting of horizontal, curved and multi-oriented images.

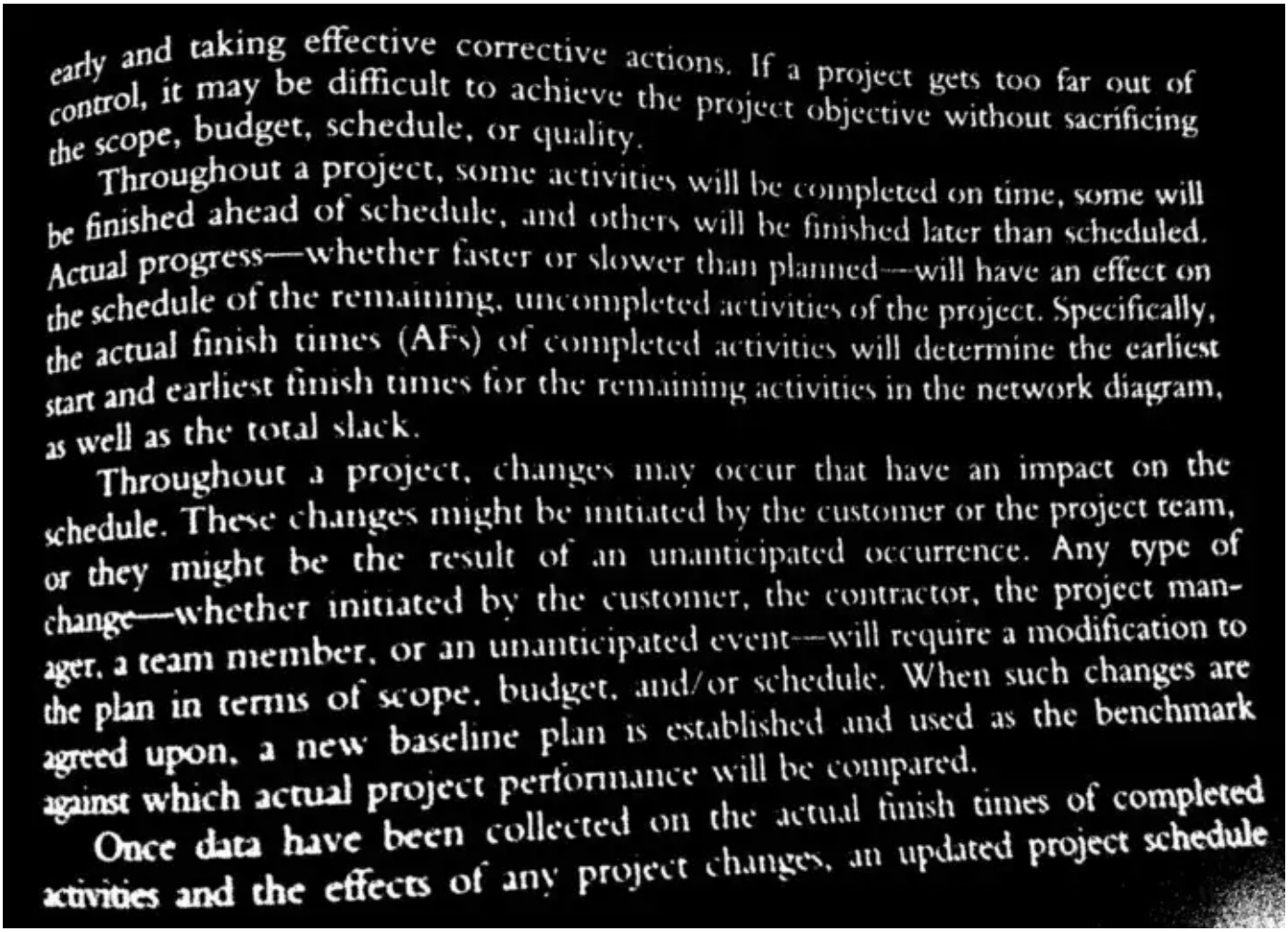
OCR

OCR or Optical Character Recognition is a system used to recognize text from a digital image or video stream. The system may include both the hardware and software components bundled together or just the software for recognition.

One popular state of the art OCR software is Tesseract. Its used to read text from images with a considerable high level of accuracy and is used in many of the pdf scanners and applications found online.

The main issue with these open source OCR on experience is that they are mostly used to treating images as documents rather than actual images found in real life scenarios. OCR's like Tesseract can work on multi-line and oriented documents but the recognition only seems to work accurately when the images are mostly documents or pages with text arranged properly.
One main reason for this being that these modules use classical computer vision techniques of binarization, thresholding etc. to try and separate out the words from

the background which in theory would be of a very high contrast to each other (think of it like a white page with black text).



early and taking effective corrective actions. If a project gets too far out of control, it may be difficult to achieve the project objective without sacrificing the scope, budget, schedule, or quality.

Throughout a project, some activities will be completed on time, some will be finished ahead of schedule, and others will be finished later than scheduled. Actual progress—whether faster or slower than planned—will have an effect on the schedule of the remaining, uncompleted activities of the project. Specifically, the actual finish times (AFs) of completed activities will determine the earliest start and earliest finish times for the remaining activities in the network diagram, as well as the total slack.

Throughout a project, changes may occur that have an impact on the schedule. These changes might be initiated by the customer or the project team, or they might be the result of an unanticipated occurrence. Any type of change—whether initiated by the customer, the contractor, the project manager, a team member, or an unanticipated event—will require a modification to the plan in terms of scope, budget, and/or schedule. When such changes are agreed upon, a new baseline plan is established and used as the benchmark against which actual project performance will be compared.

Once data have been collected on the actual finish times of completed activities and the effects of any project changes, an updated project schedule

Pre-processing Done by Tesseract before recognition

For an OCR system to actually work on text images from any image the main part is to first detect the region of text and apply the recognition algorithm on that.

Scene Text Recognition in the wild becomes a two pronged problem statement.

1. *Detect* the concerned region of text in the image.
2. Crop that detected region and apply text *recognition* algorithm on that region.

CRNN

The following is a very simple explanation of what a CRNN is to get an intuition behind the OCR training and inference.

Convolutional Recurrent Neural Networks are deep learning architectures popularly used for image-text recognition.

The **CNN** is used to extract and learn from abstract features in the image automatically. The **RNN** analyses and stores the previous information from the image features along with the current one to understand the importance of certain features during inference. Basically we are trying to sort of remember the sequence in the features too. Some ocr's use a two way RNN (Bi-LSTM) to remember the sequence more effectively and even predict future (unseen features) better.
The loss function to get the character/text, popularly seen is the **CTC** loss function. CTC loss allows the embedded RNN output to allow for variable length in the sequence, if one image is of 3 letters and the other is of 10, the CTC would predict that. Another loss function getting more popularity in the recent times is Attention primarily due to the rise of Transformer architecture in recent years and now their advent into computer vision problems.

Paddle-OCR

Paddle OCR is a lightweight ocr system with inbuilt detection and recognition in the pipeline.
It supports multilingual training and claims high inference speed along with accuracy on using mobile/edge friendly CNN architectures.

Detection visualized as bounding boxes along with the recognizer output and confidence scores.

Training

1. Convert the image data and annotations into the required format for the detector and recognizer respectively.

For this exercise will be using Total Text — Scene Text Recognition dataset from Kaggle. This dataset consists of 1555 images with more than 3 different text orientations: Horizontal, Multi-Oriented, and Curved, one of a kind.

We will download the dataset from **Kaggle** using the Kaggle API and use the Train folder and Test folder for training and evaluation respectively.
Each image contains a corresponding Json for Polygon annotations of the text region in the image along with the annotation.
Mapping each image with the json we will use only those annotations which have 8

point coordinate (4 region coordinates of x1,y1,x2,y2,x3,y3,x4,y4,x5,y5,x6,y6,x7,y7,x8,y8) since PaddleOCR only works on these.

For the **Detector** we need to create two .txt files for training and validation respectively with the format as follows:

image_path\t[{"*points*":[[x1,y1],[x2,y2],[x3,y3],[x4,y4]], " *transcription*":text_annotation}, {"points":.....}]\n

```
img478.jpg [{"points": [["500", "225"], ["681", "198"], ["682", "270"], ["483", "295"]], "transcription": "A1"}, {"po
img878.jpg [{"points": [["486", "523"], ["676", "519"], ["676", "567"], ["490", "574"]], "transcription": "AIRFREI"},
img717.jpg [{"points": [["430", "192"], ["502", "204"], ["488", "228"], ["428", "220"]], "transcription": "Play"}, {"
```

Detection txt file format

Similarly for the recognizer the two .txt file formats would be:

image_path\tAnnotation\n
image_path\tAnnotation\n

```
crnn_train/img_0.png    A1
img_1.png   ###
crnn_train/img_2.png    ALL
crnn_train/img_3.png    VEHICLES
```

Recognizer txt file format

The script for creating the training and evaluation .txt files from the Train and Test folder will be given in the notebook at the end of this article.

2. Download the models from the PaddleOCR website.

PaddleOCR provides multiple pre-trained models for both English, Chinese, Devnagiri and other languages. Check the git repository for the language you want to train your ocr on.

Here we are using **MobileNet_V3_Large** backend for the detector and their latest **PP-OCR-V3** model for the recognizer respectively.

```
In [19]: !wget -P ./pretrain_models/ https://paddleocr.bj.bcebos.com/pretrained/MobileNetV3_large_x0_5_pretrained.pdparams

--2022-08-14 15:45:39--  https://paddleocr.bj.bcebos.com/pretrained/MobileNetV3_large_x0_5_pretrained.pdparams
Resolving paddleocr.bj.bcebos.com (paddleocr.bj.bcebos.com)... 103.235.46.61, 2409:8c04:1001:1002:0:ff:b001:368a
Connecting to paddleocr.bj.bcebos.com (paddleocr.bj.bcebos.com)|103.235.46.61|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3189601 (3.0M) [application/octet-stream]
Saving to: ‘./pretrain_models/MobileNetV3_large_x0_5_pretrained.pdparams’

MobileNetV3_large_x 100%[===================>]   3.04M   587KB/s    in 8.7s

2022-08-14 15:45:49 (360 KB/s) - ‘./pretrain_models/MobileNetV3_large_x0_5_pretrained.pdparams’ saved [3189601/318960
1]
```

Downloading the Detector weights for training

```
# Download the pre-trained model of en_PP-OCRv3
!wget -P ./pretrain_models/ https://paddleocr.bj.bcebos.com/PP-OCRv3/english/en_PP-OCRv3_rec_train.tar
# Decompress model parameters
%cd pretrain_models
!tar -xf en_PP-OCRv3_rec_train.tar && rm -rf en_PP-OCRv3_rec_train.tar
%cd ..
```

Downloading the Recognizer weights for training

3. Change the .yml config files

Now that the annotations and images are ready we need to edit the config files for both the detector and recognizer by adding our own local paths.

Once you have cloned the PaddleOCR repository before downloading the weights.

For **detection** in the *PaddleOCR/configs/det/det_mv3_db.yml*
Under *Global.pretrained_model* add the path of the weights you have downloaded them.
Under *Train.dataset.data_dir* and *Train.dataset.label_file_list* add the path of Training images folder and the training .txt file path respectively
Under *Eval.dataset.data_dir* and *Eval.dataset.label_file_list* add the path of Evaluation images folder and the evaluation.txt file path respectively

```
Eval:
  dataset:
    name: SimpleDataSet
    data_dir: crnn_test/
    label_file_list:
    - rec_test.txt
```

replace this with your own paths

For **recognition** in the *PaddleOCR/configs/rec/PP-OCRv3/en_PP-OCRv3_rec.yml* repeat the same edits as earlier.

4. Start Training

```
#Multi-GPU Training
!python3 -m paddle.distributed.launch --gpus '0,1'  tools/train.py -c configs/det/det_mv3_db.yml
```

Detector Training Script

```
#Single GPU Training
!python3 tools/train.py -c configs/det/det_mv3_db.yml  \
        -o Global.pretrained_model=./pretrain_models/MobileNetV3_large_x0_5_pretrained
```

Open in app Sign up Sign In

Search Medium

Recognizer Training Script

Inference

1. Conversion of weights

After Training the weights we need to convert or export them before we can run inference.

```
#conversion of Detection Trained Weights to Inference

!python3 tools/export_model.py -c configs/det/det_mv3_db.yml -o Global.pretrained_model="./output/db_mv3/best_accuracy" Global.save_inference_dir="./output/det_db_inference/"

W0821 04:14:38.911141 14130 gpu_resources.cc:61] Please NOTE: device: 0, GPU Compute Capability: 7.5, Driver API Version: 11.4, Runtime API Version: 11.2
W0821 04:14:38.917029 14130 gpu_resources.cc:91] device: 0, cuDNN Version: 8.2.
[2022/08/21 04:14:41] ppocr INFO: load pretrain successful from ./output/db_mv3/best_accuracy
[2022/08/21 04:14:44] ppocr INFO: inference model is saved to ./output/det_db_inference/inference
```

Detector weights export

python3 tools/export_model.py -c configs/det/det_mv3_db.yml -o Global.pretrained_model="./output/db_mv3/best_accuracy" Global.save_inference_dir="./output/det_db_inference/"

```
#Conversion of Recognizer Trained weights to Inference

!python3 tools/export_model.py -c configs/rec/PP-OCRv3/en_PP-OCRv3_rec.yml -o Global.pretrained_model=output/v3_en_mobile/best_accuracy  Global.save_inference_dir=./inference/en_PP-OCRv3_rec/

W0821 04:11:47.627471 10865 gpu_resources.cc:61] Please NOTE: device: 0, GPU Compute Capability: 7.5, Driver API Version: 11.4, Runtime API Version: 11.2
W0821 04:11:47.658488 10865 gpu_resources.cc:91] device: 0, cuDNN Version: 8.2.
[2022/08/21 04:11:50] ppocr INFO: load pretrain successful from output/v3_en_mobile/best_accuracy
[2022/08/21 04:11:52] ppocr INFO: inference model is saved to ./inference/en_PP-OCRv3_rec/inference
```

Recognizer weights conversion

python3 tools/export_model.py -c configs/rec/PP-OCRv3/en_PP-OCRv3_rec.yml -o Global.pretrained_model=output/v3_en_mobile/best_accuracy Global.save_inference_dir=./inference/en_PP-OCRv3_rec/

2. Loading the weights using the PaddleOCR Library

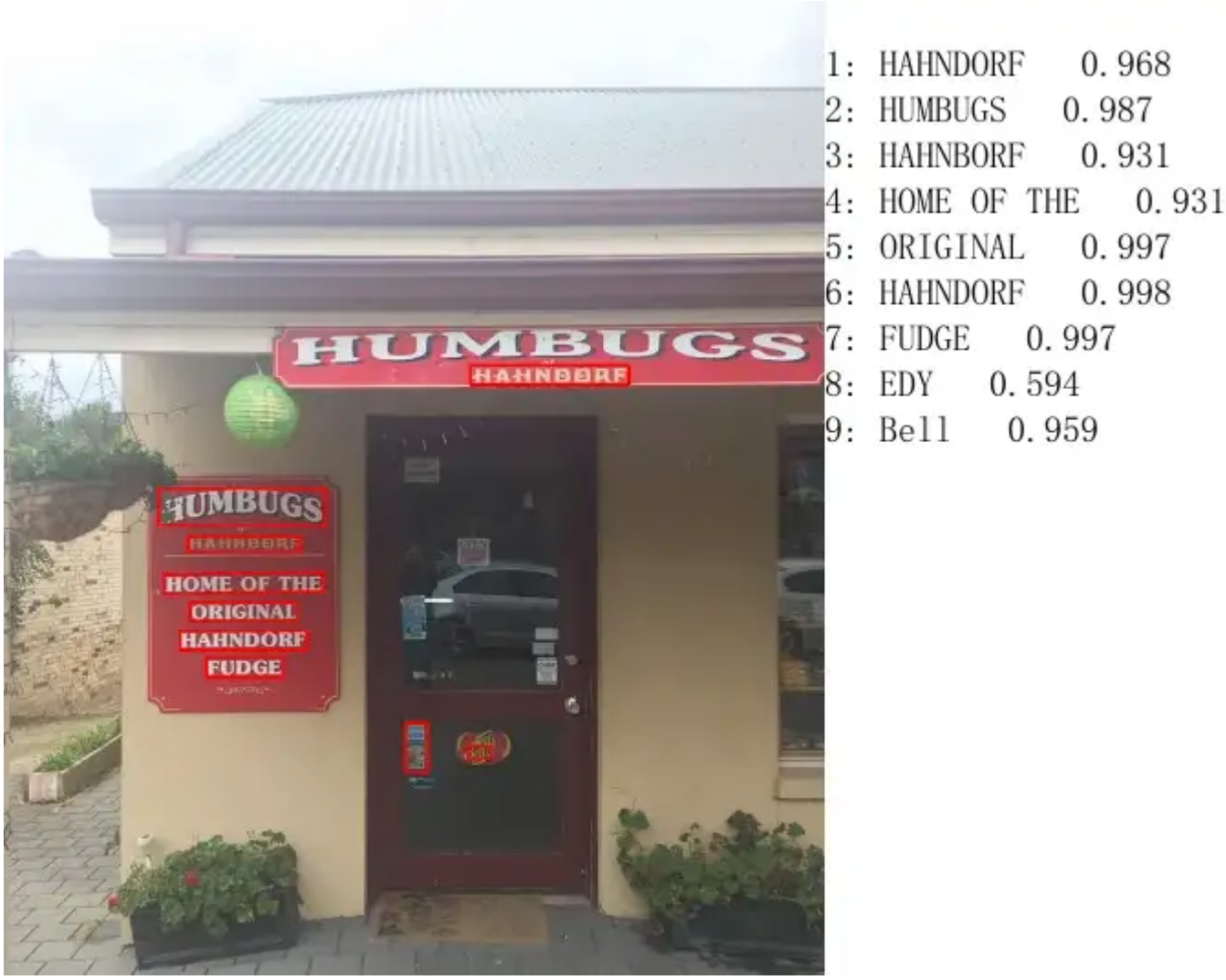
```
#Loading the detector and ocr with the previously trained weights
custom_ocr = PaddleOCR(use_angle_cls=True,
                       rec_model_dir='PaddleOCR/inference/en_PP-OCRv3_rec',
                       det_model_dir='PaddleOCR/output/det_db_inference',
                       rec_char_d                    utils/en_dict.txt',
                       use_gpu=Tr
                       show_log=False)
```

14 |

We will take a sample image and load it using OpenCV which we'll give it to the OCR.

Testing Image

Finally with our results we will draw the bounding boxes and print the ocr results along with the confidence scores for each box respectively.



Final Results

Note

1. Cpu users pip install paddlepaddle while GPU users pip install paddlepaddle-gpu

2. While running on Colab the installation works fine but using a gpu with CUDNN >8.0 gives an error. Any error while training or loading the model which are shown as Segmentation Error or C++ Stack error is incompatibility of the paddlepaddle-gpu with your card's CUDNN version.
   Resolved this by using their official website before installing the library in a fresh python3 environment.

Using CUDA 11.4 and CUDNN 8.2 the above installation worked for me

Notebook Link

Prequisites

In [1]:

```
import cv2
import os
#from imutils import perspective
import numpy as np
import json
from tqdm import tqdm
import matplotlib.pyplot as plt
from imutils import perspective
```

Detector Training

In [11]:

```
print("Number of Training images are", len(os.listdir("Total-Text")))
```

```
Number of Training images are 1255
```

In [12]:

```
# Preparing Training Annotation Text file

count = 0
flag = 1
```

Paddle_OCR_Training.ipynb hosted with ❤ by **GitHub** **view raw**

https://gist.github.com/leonbora167/049ac6622b7a2fb5c23ec48070af486f

Ocr Paddleocr Deep Learning Machine Learning Computer Vision

About Help Terms Privacy

Get the Medium app